Project Executable Files

Date	15 March 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	No
8	Dockerfile / Containerization config (if applicable)	No
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
smart-Lender-/
├── data/
|   ├── test_Y3wMUE5_7gLdaTN.csv
|   └── train_u6lujuX_CVtuZ9i.csv
├── docs/
|   ├── screenshots/
|   ├── Performance_Testing_Filled_Template.html
|   ├── architecture_diagram.md
|   ├── automated_tester.py
|   ├── model_comparison.md
|   ├── performance_testing_filled.md
|   ├── performance_testing_report.md
|   ├── test_results_all.csv
|   └── testing_report.md
├── models/
|   ├── feature_names.pkl
|   └── loan_model.pkl
├── notebooks/
|   └── 01_EDA.ipynb
├── static/
|   ├── style.css
|   └── style_backup.css
├── templates/
|   ├── 403.html
|   ├── 404.html
|   ├── admin_dashboard.html
|   ├── all_predictions.html
|   ├── analyst_dashboard.html
|   ├── base.html
|   ├── index.html
|   ├── index_backup.html
|   ├── login.html
|   ├── manage_users.html
|   ├── officer_dashboard.html
|   ├── prediction_history.html
|   ├── privacy.html
|   └── register.html
├── .gitignore
├── README.md
├── app.py
├── loan_predictions.csv
├── requirements.txt
└── train_model.py
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://smart-lender-n96s.onrender.com/
Login Credentials (Demo)	Login Credentials (Demo): 1. System Administrator Access - Email: admin@demo.com - Password: AdminPassword123! - Role: Admin

	2. Loan Analyst Access - Email: analyst@demo.com - Password: AnalystPassword123! - Role: Analyst 3. Loan Officer Access - Email: officer@demo.com - Password: OfficerPassword123! - Role: Officer
Platform / Hosting Provider	Render
Repository Link	https://github.com/pushpa-sree/smart-Lender-.git
Demo Video Link	

Step 4: Run Instructions

Pre-requisites:

- Python 3.8 or higher installed
- pip (Python package installer)
- Git (optional, if cloning from a repository)

Step 1: Extract the project folder or clone the repository to your local machine

Step 2: Open a terminal or command prompt and navigate to the root directory of the project (smart-Lender-).

Step 3: (Recommended) Create and activate a virtual environment:

- Windows: python -m venv venv then venv\Scripts\activate
- Mac/Linux: python3 -m venv venv then source venv/bin/activate

Step 4: Install the required dependencies by running: pip install -r requirements.txt

Step 5: Start the Flask web application by running: python app.py *(Note: The database tables are automatically initialized during startup.)*

Step 6: Open your web browser and navigate to: http://127.0.0.1:5000

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	**No Default Users pre-seeded in Database** The SQLite database initializes empty without default demo users.	**Workaround:** Evaluators must manually register the demo accounts via the /register page before attempting to log in with them.
2	**Local SQLite Database Concurrency** The app uses a local SQLite database which does not handle high-concurrency write operations well.	**Status:** Perfectly fine for local evaluation and demonstration. For a production environment, it should be migrated to PostgreSQL or MySQL.
3	**n-Memory Model Loading** The ML model (loan_model.pkl) is loaded entirely into RAM when the Flask app starts.	**Status:** Acceptable for this project size. In a real-world enterprise setting, the model would be decoupled using a dedicated serving API (e.g., Flask-RESTful microservice).